```sql
-- ============================================================
-- Database Assignment 06 - SQL Questions
-- Database: Hospital (restore it from Hospital.bak before running)
-- ============================================================

USE Hospital;
GO

-- ------------------------------------------------------------
-- Question 1 - Count Patients per Ward
-- Display the number of patients assigned to each ward.
-- ------------------------------------------------------------
SELECT WardId,
       COUNT(*) AS PatientCount
FROM Patients
WHERE WardId IS NOT NULL
GROUP BY WardId;
GO

-- ------------------------------------------------------------
-- Question 2 - Calculate the Average Consultant Salary
-- Display the average salary of all consultants.
-- ------------------------------------------------------------
SELECT AVG(Salary) AS AverageSalary
FROM Consultants;
GO

-- ------------------------------------------------------------
-- Question 3 - Display Salary Statistics for Consultants
-- Display the minimum, maximum, average, and total consultant salaries.
-- ------------------------------------------------------------
SELECT MIN(Salary) AS MinimumSalary,
       MAX(Salary) AS MaximumSalary,
       AVG(Salary) AS AverageSalary,
       SUM(Salary) AS TotalSalary
FROM Consultants;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 4 - Count Patients by Ward
-- Display the ward ID and number of patients in each ward, sorted from highest to lowest.
-- ------------------------------------------------------------
SELECT WardId,
       COUNT(*) AS PatientCount
FROM Patients
WHERE WardId IS NOT NULL
GROUP BY WardId
ORDER BY PatientCount DESC;
GO

-- ------------------------------------------------------------
-- Question 5 - Medication Quantity per Patient
-- Display the total medication quantity administered to each patient.
-- ------------------------------------------------------------
SELECT PatientId,
       SUM(Quantity) AS TotalQuantity
FROM DrugAdministrations
GROUP BY PatientId;
GO

-- ------------------------------------------------------------
-- Question 6 - Medication Administrations per Patient
-- Display each patient ID together with the number of medication administrations they received.
-- ------------------------------------------------------------
SELECT PatientId,
       COUNT(*) AS AdministrationCount
FROM DrugAdministrations
GROUP BY PatientId;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 7 - Patients per Ward with Ward Names
-- Display every ward together with the number of patients assigned to it, including wards with
-- no patients.
-- COUNT(p.Id) skips the NULL row the LEFT JOIN adds for an empty ward, so it counts 0, not 1.
-- ------------------------------------------------------------
SELECT w.Id        AS WardId,
       w.Name      AS WardName,
       COUNT(p.Id) AS PatientCount
FROM Wards AS w
LEFT JOIN Patients AS p ON p.WardId = w.Id
GROUP BY w.Id, w.Name;
GO

-- ------------------------------------------------------------
-- Question 8 - Average Nurse Salary per Ward
-- Display each ward together with the average salary of nurses serving in that ward.
-- ------------------------------------------------------------
SELECT w.Id          AS WardId,
       w.Name        AS WardName,
       AVG(n.Salary) AS AverageNurseSalary
FROM Wards AS w
INNER JOIN Nurses AS n ON n.ServesInWardId = w.Id
GROUP BY w.Id, w.Name;
GO

-- ------------------------------------------------------------
-- Question 9 - Wards with More Than 3 Patients
-- Display wards that have more than 3 patients.
-- ------------------------------------------------------------
SELECT w.Id        AS WardId,
       w.Name      AS WardName,
       COUNT(p.Id) AS PatientCount
FROM Wards AS w
INNER JOIN Patients AS p ON p.WardId = w.Id
GROUP BY w.Id, w.Name
HAVING COUNT(p.Id) > 3;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 10 - Patients with More Than 5 Medication Administrations
-- Display patients who received medication more than 5 times.
-- ------------------------------------------------------------
SELECT p.Id     AS PatientId,
       p.Name   AS PatientName,
       COUNT(*) AS AdministrationCount
FROM Patients AS p
INNER JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name
HAVING COUNT(*) > 5;
GO

-- ------------------------------------------------------------
-- Question 11 - Consultants Above Average Salary
-- Display consultants whose salary is greater than the average consultant salary.
-- ------------------------------------------------------------
SELECT Id,
       Name,
       Salary
FROM Consultants
WHERE Salary > (SELECT AVG(Salary) FROM Consultants);
GO

-- ------------------------------------------------------------
-- Question 12 - Nurses Above Average Salary
-- Display nurses whose salary is greater than the average nurse salary.
-- ------------------------------------------------------------
SELECT Number,
       Name,
       Salary
FROM Nurses
WHERE Salary > (SELECT AVG(Salary) FROM Nurses);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 13 - Patients in the Largest Ward
-- Display patients who belong to the ward containing the largest number of patients.
-- WITH TIES keeps every ward that shares the largest count, if there is a tie.
-- ------------------------------------------------------------
SELECT Id,
       Name,
       DOB,
       WardId
FROM Patients
WHERE WardId IN (
    SELECT TOP 1 WITH TIES WardId
    FROM Patients
    WHERE WardId IS NOT NULL
    GROUP BY WardId
    ORDER BY COUNT(*) DESC
);
GO

-- ------------------------------------------------------------
-- Question 14 - Consultants with the Maximum Salary
-- Display the consultant or consultants who have the highest salary.
-- ------------------------------------------------------------
SELECT Id,
       Name,
       Salary
FROM Consultants
WHERE Salary = (SELECT MAX(Salary) FROM Consultants);
GO

-- ------------------------------------------------------------
-- Question 15 - Patients Who Received Medication
-- Display patients who have at least one medication administration.
-- ------------------------------------------------------------
SELECT p.Id,
       p.Name
FROM Patients AS p
WHERE EXISTS (SELECT 1 FROM DrugAdministrations AS da WHERE da.PatientId = p.Id);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 16 - Patients Who Never Received Medication
-- Display patients who have never received any medication.
-- ------------------------------------------------------------
SELECT p.Id,
       p.Name
FROM Patients AS p
WHERE NOT EXISTS (SELECT 1 FROM DrugAdministrations AS da WHERE da.PatientId = p.Id);
GO

-- ------------------------------------------------------------
-- Question 17 - Count Medication Administrations per Patient
-- Display every patient together with the number of medication administrations they received.
-- ------------------------------------------------------------
SELECT p.Id                AS PatientId,
       p.Name              AS PatientName,
       COUNT(da.PatientId) AS AdministrationCount
FROM Patients AS p
LEFT JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name;
GO

-- ------------------------------------------------------------
-- Question 18 - Total Medication Quantity per Patient
-- Display every patient together with the total medication quantity administered to them.
-- ------------------------------------------------------------
SELECT p.Id                           AS PatientId,
       p.Name                         AS PatientName,
       COALESCE(SUM(da.Quantity), 0) AS TotalQuantity
FROM Patients AS p
LEFT JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 19 - Derived Table for Consultant Salaries
-- Create a derived table containing the average consultant salary, then display consultants
-- whose salary is above that average.
-- ------------------------------------------------------------
SELECT c.Id,
       c.Name,
       c.Salary,
       a.AverageSalary
FROM Consultants AS c
CROSS JOIN (SELECT AVG(Salary) AS AverageSalary FROM Consultants) AS a
WHERE c.Salary > a.AverageSalary;
GO

-- ------------------------------------------------------------
-- Question 20 - Patients with Above-Average Medication Quantity
-- Display patients whose total medication quantity is greater than the average total medication
-- quantity across patients who received medication.
-- AVG over INT values returns an INT in SQL Server, so the totals are cast to keep the decimals.
-- ------------------------------------------------------------
WITH PatientTotals AS (
    SELECT PatientId,
           SUM(Quantity) AS TotalQuantity
    FROM DrugAdministrations
    GROUP BY PatientId
)
SELECT p.Id   AS PatientId,
       p.Name AS PatientName,
       pt.TotalQuantity
FROM PatientTotals AS pt
INNER JOIN Patients AS p ON p.Id = pt.PatientId
WHERE pt.TotalQuantity > (SELECT AVG(CAST(TotalQuantity AS DECIMAL(10, 2))) FROM PatientTotals);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 21 - Create a Patient Backup
-- Create a new table containing all patients.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS PatientsBackup;

SELECT *
INTO PatientsBackup
FROM Patients;
GO

-- ------------------------------------------------------------
-- Question 22 - Create a Basic Patient Information Table
-- Create a new table containing only patient ID, name, and date of birth.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS PatientBasicInfo;

SELECT Id,
       Name,
       DOB
INTO PatientBasicInfo
FROM Patients;
GO

-- ------------------------------------------------------------
-- Question 23 - Create a Consultant Salary Report
-- Create a new table containing consultant ID, name, and salary.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS ConsultantSalaryReport;

SELECT Id,
       Name,
       Salary
INTO ConsultantSalaryReport
FROM Consultants;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 24 - Create a Patient-Ward Report
-- Create a new table containing each patient together with the ward they belong to.
-- LEFT JOIN keeps the patients who are not assigned to a ward.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS PatientWardReport;

SELECT p.Id   AS PatientId,
       p.Name AS PatientName,
       w.Name AS WardName
INTO PatientWardReport
FROM Patients AS p
LEFT JOIN Wards AS w ON w.Id = p.WardId;
GO

-- ------------------------------------------------------------
-- Question 25 - Create a Patient-Consultant Report
-- Create a new table containing patients and their assigned consultants.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS PatientConsultantReport;

SELECT p.Id   AS PatientId,
       p.Name AS PatientName,
       c.Id   AS ConsultantId,
       c.Name AS ConsultantName
INTO PatientConsultantReport
FROM PatientConsultantAssignments AS pca
INNER JOIN Patients AS p ON p.Id = pca.PatientId
INNER JOIN Consultants AS c ON c.Id = pca.ConsultantId;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 26 - Create a High-Salary Consultant Table
-- Create a new table containing consultants whose salary is greater than 50000.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS HighSalaryConsultants;

SELECT Id,
       Name,
       Salary
INTO HighSalaryConsultants
FROM Consultants
WHERE Salary > 50000;
GO

-- ------------------------------------------------------------
-- Question 27 - Create a Young Patients Table
-- Create a new table containing patients born after January 1, 2000.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS YoungPatients;

SELECT *
INTO YoungPatients
FROM Patients
WHERE DOB > '2000-01-01';
GO
```

```sql
-- ------------------------------------------------------------
-- Question 28 - Create a High-Salary Nurse Report
-- Create a new table containing nurses earning more than 30000 together with the ward they serve
-- in.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS HighSalaryNurses;

SELECT n.Number,
       n.Name,
       n.Salary,
       w.Name AS WardName
INTO HighSalaryNurses
FROM Nurses AS n
LEFT JOIN Wards AS w ON w.Id = n.ServesInWardId
WHERE n.Salary > 30000;
GO

-- ------------------------------------------------------------
-- Question 29 - Create a Ward Patient Summary
-- Create a new table containing each ward and its patient count.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS WardPatientSummary;

SELECT w.Id        AS WardId,
       w.Name      AS WardName,
       COUNT(p.Id) AS PatientCount
INTO WardPatientSummary
FROM Wards AS w
LEFT JOIN Patients AS p ON p.WardId = w.Id
GROUP BY w.Id, w.Name;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 30 - Create a Patient Medication Summary
-- Create a new table containing each patient and their total medication quantity.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS PatientMedicationSummary;

SELECT p.Id                          AS PatientId,
       p.Name                        AS PatientName,
       COALESCE(SUM(da.Quantity), 0) AS TotalQuantity
INTO PatientMedicationSummary
FROM Patients AS p
LEFT JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name;
GO

-- ------------------------------------------------------------
-- Question 31 - Calculate Patient Age
-- Create a scalar function that receives a date of birth and returns the patient's age.
-- One year is taken off when this year's birthday has not arrived yet,
-- because DATEDIFF(YEAR, ...) only compares the year parts.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.CalculatePatientAge (@DateOfBirth DATE)
RETURNS INT
AS
BEGIN
    RETURN DATEDIFF(YEAR, @DateOfBirth, GETDATE())
         - CASE
               WHEN DATEADD(YEAR, DATEDIFF(YEAR, @DateOfBirth, GETDATE()), @DateOfBirth) > CAST(GETDATE() AS DATE)
               THEN 1
               ELSE 0
           END;
END;
GO

SELECT dbo.CalculatePatientAge('1990-05-15') AS Age;
GO
```

```sql
-- ------------------------------------------------------------
-- Question 32 - Calculate Annual Salary
-- Create a scalar function that receives a monthly salary and returns the annual salary.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.CalculateAnnualSalary (@MonthlySalary DECIMAL(18, 2))
RETURNS DECIMAL(18, 2)
AS
BEGIN
    RETURN @MonthlySalary * 12;
END;
GO

SELECT dbo.CalculateAnnualSalary(15000) AS AnnualSalary;
GO

-- ------------------------------------------------------------
-- Question 33 - Calculate Medication Cost
-- Create a scalar function that receives quantity and unit price and returns the total
-- medication cost.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.CalculateMedicationCost (@Quantity INT, @UnitPrice DECIMAL(10, 2))
RETURNS DECIMAL(18, 2)
AS
BEGIN
    RETURN @Quantity * @UnitPrice;
END;
GO

SELECT dbo.CalculateMedicationCost(3, 12.50) AS MedicationCost;
GO
```

Question 34

```sql
-- ------------------------------------------------------------
-- Question 34 - Get Patients by Ward
-- Create an inline table-valued function that receives a ward ID and returns all patients in
-- that ward.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.GetPatientsByWard (@WardId INT)
RETURNS TABLE
AS
RETURN
(
    SELECT Id,
           Name,
           DOB,
           WardId
    FROM Patients
    WHERE WardId = @WardId
);
GO

SELECT * FROM dbo.GetPatientsByWard(1);
GO
```

Question 35

```sql
-- ------------------------------------------------------------
-- Question 35 - Get Consultants by Minimum Salary
-- Create an inline table-valued function that receives a minimum salary and returns consultants
-- whose salary is greater than or equal to it.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.GetConsultantsByMinimumSalary (@MinimumSalary DECIMAL(18, 0))
RETURNS TABLE
AS
RETURN
(
    SELECT Id,
           Name,
           Salary
    FROM Consultants
    WHERE Salary >= @MinimumSalary
);
GO

SELECT * FROM dbo.GetConsultantsByMinimumSalary(20000);
GO
```

Question 36


```sql
-- ------------------------------------------------------------
-- Question 36 - Get Patients with Their Ward
-- Create an inline table-valued function that receives a ward ID and returns patients together
-- with their ward name.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.GetPatientsWithWard (@WardId INT)
RETURNS TABLE
AS
RETURN
(
    SELECT p.Id,
           p.Name,
           p.DOB,
           w.Name AS WardName
    FROM Patients AS p
    INNER JOIN Wards AS w ON w.Id = p.WardId
    WHERE p.WardId = @WardId
);
GO

SELECT * FROM dbo.GetPatientsWithWard(2);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 37 - Get Patients Above a Specific Age
-- Create a multi-statement table-valued function that receives a minimum age and returns
-- patients who are at least that age.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.GetPatientsWithAge (@MinimumAge INT)
RETURNS @Patients TABLE
(
    Id   INT,
    Name NVARCHAR(100),
    DOB  DATE,
    Age  INT
)
AS
BEGIN
    INSERT INTO @Patients (Id, Name, DOB, Age)
    SELECT Id,
           Name,
           DOB,
           dbo.CalculatePatientAge(DOB)
    FROM Patients
    WHERE dbo.CalculatePatientAge(DOB) >= @MinimumAge;

    RETURN;
END;
GO

SELECT * FROM dbo.GetPatientsWithAge(60);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 38 - Get Patient Medication Summary
-- Create a multi-statement table-valued function that receives a patient ID and returns the
-- total medication quantity and number of medication administrations.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.GetPatientMedicationSummary (@PatientId INT)
RETURNS @Summary TABLE
(
    PatientId           INT,
    TotalQuantity       INT,
    AdministrationCount INT
)
AS
BEGIN
    INSERT INTO @Summary (PatientId, TotalQuantity, AdministrationCount)
    SELECT @PatientId,
           COALESCE(SUM(Quantity), 0),
           COUNT(*)
    FROM DrugAdministrations
    WHERE PatientId = @PatientId;

    RETURN;
END;
GO

SELECT * FROM dbo.GetPatientMedicationSummary(201);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 39 - Classify Consultant Salaries
-- Create a multi-statement table-valued function that receives a minimum salary and returns
-- consultants with a salary classification.
-- ------------------------------------------------------------
CREATE OR ALTER FUNCTION dbo.ClassifyConsultantSalaries (@MinimumSalary DECIMAL(18, 0))
RETURNS @Consultants TABLE
(
    Id                   INT,
    Name                 NVARCHAR(100),
    Salary               DECIMAL(18, 0),
    SalaryClassification NVARCHAR(20)
)
AS
BEGIN
    INSERT INTO @Consultants (Id, Name, Salary, SalaryClassification)
    SELECT Id,
           Name,
           Salary,
           CASE
               WHEN Salary >= 20000 THEN 'High'
               WHEN Salary >= 15000 THEN 'Medium'
               ELSE 'Low'
           END
    FROM Consultants
    WHERE Salary >= @MinimumSalary;

    RETURN;
END;
GO

SELECT * FROM dbo.ClassifyConsultantSalaries(15000);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 40 - Patient Medication Statistics
-- Display each patient together with their number of medication administrations, total
-- medication quantity, and average medication quantity.
-- ------------------------------------------------------------
SELECT p.Id                                          AS PatientId,
       p.Name                                        AS PatientName,
       COUNT(da.PatientId)                           AS AdministrationCount,
       COALESCE(SUM(da.Quantity), 0)                 AS TotalQuantity,
       AVG(CAST(da.Quantity AS DECIMAL(10, 2))) AS AverageQuantity
FROM Patients AS p
LEFT JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name;
GO

-- ------------------------------------------------------------
-- Question 41 - Above-Average Wards
-- Display wards whose patient count is greater than the average patient count across all wards.
-- Wards with no patients count as 0, because the average is taken across all wards.
-- ------------------------------------------------------------
WITH WardCounts AS (
    SELECT w.Id        AS WardId,
           w.Name      AS WardName,
           COUNT(p.Id) AS PatientCount
    FROM Wards AS w
    LEFT JOIN Patients AS p ON p.WardId = w.Id
    GROUP BY w.Id, w.Name
)
SELECT WardId,
       WardName,
       PatientCount
FROM WardCounts
WHERE PatientCount > (SELECT AVG(CAST(PatientCount AS DECIMAL(10, 2))) FROM WardCounts);
GO
```

```sql
-- ------------------------------------------------------------
-- Question 42 - Create a High-Activity Patient Report
-- Create a new table containing patients who received medication more than 5 times.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS HighActivityPatients;

SELECT p.Id     AS PatientId,
       p.Name   AS PatientName,
       COUNT(*) AS AdministrationCount
INTO HighActivityPatients
FROM Patients AS p
INNER JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name
HAVING COUNT(*) > 5;
GO

-- ------------------------------------------------------------
-- Question 43 - Create a Consultant Salary Report Using a Scalar Function
-- Create a new table containing each consultant's name, monthly salary, and annual salary.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS ConsultantAnnualSalaryReport;

SELECT Name,
       Salary                              AS MonthlySalary,
       dbo.CalculateAnnualSalary(Salary) AS AnnualSalary
INTO ConsultantAnnualSalaryReport
FROM Consultants;
GO

-- ------------------------------------------------------------
-- Question 44 - Use an Inline TVF with a Filter
-- Using GetConsultantsByMinimumSalary, display consultants whose salary is at least 50000 and
-- whose name starts with A.
-- ------------------------------------------------------------
SELECT Id,
       Name,
       Salary
FROM dbo.GetConsultantsByMinimumSalary(50000)
WHERE Name LIKE 'A%';
GO
```

```sql
-- ------------------------------------------------------------
-- Question 45 - Use a Multi-Statement TVF
-- Using GetPatientsWithAge, display patients who are at least 30 years old and order them from
-- oldest to youngest.
-- ------------------------------------------------------------
SELECT Id,
       Name,
       DOB,
       Age
FROM dbo.GetPatientsWithAge(30)
ORDER BY Age DESC, DOB;
GO

-- ------------------------------------------------------------
-- Question 46 - Final Hospital Patient Summary
-- Display patient ID, patient name, ward name, patient age, number of medication
-- administrations, and total medication quantity. Use the existing tables and
-- CalculatePatientAge.
-- ------------------------------------------------------------
SELECT p.Id                            AS PatientId,
       p.Name                          AS PatientName,
       w.Name                          AS WardName,
       dbo.CalculatePatientAge(p.DOB) AS Age,
       COUNT(da.PatientId)             AS AdministrationCount,
       COALESCE(SUM(da.Quantity), 0)  AS TotalQuantity
FROM Patients AS p
LEFT JOIN Wards AS w ON w.Id = p.WardId
LEFT JOIN DrugAdministrations AS da ON da.PatientId = p.Id
GROUP BY p.Id, p.Name, w.Name, p.DOB;
GO
```